1. Tree-split cut-out screen on structured factor bases — run report

Study `research/ic_tree_split_structured_v_20261011` (aburan28/crypto). **Run** `runs/registered`, started 2026-10-11T21:05:51Z, last cell 21:10:11Z (UTC). **Registration** commit `3dbbfa1d` (pushed before the run); cairn registration objective `sha256:f236a122…`. **Run commit** `dbd878f0`; harness binary sha256 `069f497e…7828`.

1.1. What ran

`run.sh`: `examples/tree_split_structured.rs` (native Rust), one invocation per cell, 64 cells requested, **64 completed**, 0 failed, 0 censored (`runs/registered/cells.log`). Host: Claude Code cloud container, 4 vCPU, 15 GB; no isolation earned or needed — the metric is a degree and the study is untimed (per-degree `secs` are logged, not measurements). Harness self-test passed before the cells (`selftest.txt`).

Arms (curve `K_a: y² + xy = x³ + a x² + 1` over `GF(2^n)`, factor base `F = {P : x(P) ∈ V\{0}}`, `T2 = {x(P+Q)}`):

- `poly`: `V = span{1, z, …, z^(ℓ−1)}`, n ∈ {17, 19, 21}, a ∈ {0, 1}, seed 20261011, ℓ = 4…8 (30 cells).
- `frob`: `V = ker g(σ), g | t^21 − 1`, not a subfield; seeds 20261011, 20261012 (20 cells) + n = 17, ℓ = 8 supplementary (2).
- `rand`: random `V`, n = 21 (10 cells, replication of the 2026-09-30 study). `subfield`: `V = GF(2^7) ⊂ GF(2^21)` (2 cells, gate).

Control per cell: uniform random subset of `GF(2)^n\{0}` of size `|T2|`. Metric: sampled cut-out degree `D_s` (K = 2000 samples, D ≤ 6, ≤ 90,000 monomials).

1.2. Recorded values

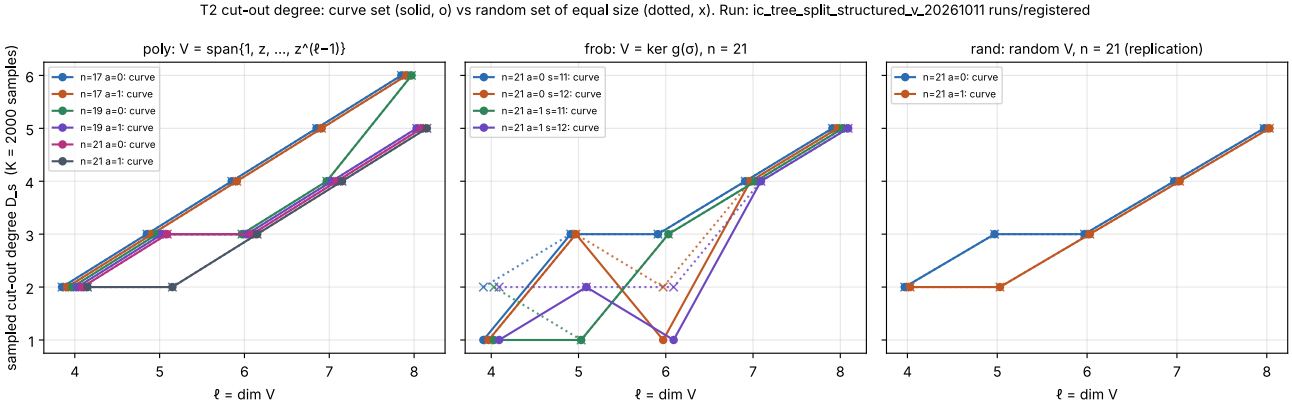


Figure 1: Measured `D_s` against ℓ; solid = curve `T2`, dotted = random control of equal size; one colour per series (n, a, seed). Run `runs/registered`, 2000 samples per arm per degree. Source: `cells.jsonl`; plotter `reports/plot_cells.py`.

arm	series	cells	gaps D_s(curve) − D_s(R)	slopes of D_s(curve) on ℓ	registered verdict
poly	6	30	all 0	0.7 – 1.0	**CLOSED**
frob (n = 21)	4	20	14 × 0, 6 × −1	0.9 – 1.1	**CLOSED**
rand (n = 21)	2	10	all 0	0.7, 0.8	**CLOSED**
subfield gate	2	2	T2 ⊆ GF(2^7); deg-1 deficit 14, 14 (≥ 14 needed)	—	**PASS**

Gaps and slopes are derived from the recorded `D_s` by the registered rule (ALIVE needs gap ≤ −2 at top ℓ in every series and lower-median slope ≤ 0.15). The six `frob` cells at −1 are exactly those whose `g` divides `t^7 − 1` (g = 23, 29, 127), so `V ⊂ GF(2^7)` — subfield leakage, identified post hoc (`lean/TreeSplit/Leak.lean`); every other `frob` cell has gap 0.

1.3. Checks on the result

- **Lean, kernel** (`Verdict.lean`, `decide`): the registered rule (`Rule.lean`, sha256 `714601c1…760d`) applied to the 64 recorded cells gives gate = true, poly = frob = rand = CLOSED. Mutating the claimed verdict fails the build.
- **Lean, compiler-trusted** (`Replay.lean`, `native_decide`): an independent implementation (highest-bit pivot elimination; harness uses lowest) recomputes `|T2|`, `D_s(curve)`, `D_s(R)` for the 12 poly cells with ℓ ≤ 5: 12/12 exact.
- **cairn** (`cairn/log.jsonl`, 19 entries, epochs 5 s): registration objective accepted before the run; result objective `sha256:2fb64f9a…` accepted by the kernel; replay objective `sha256:d1fd6832…` rejected (cairn refuses `native_decide`). `cairn audit`: chain intact, settled claims re-verified.
- **CI**: `.github/workflows/tree-split-structured-v-lean.yml` regenerates `Data.lean` from the records, checks the rule hash, and runs `lake build`.

1.4. Scope

Two Koblitz curves, $n \le 21$, $\ell \le 8$, T2 only (not Z), a sampled screen. Not tested: normal-basis windows, other bases, `Z ∩ A_R`, larger sizes. No exponent, speedup or deployed-curve claim.

Raw records: `runs/registered/{cells.jsonl, cells.log, readout.txt, selftest.txt, commit, sha256}`.